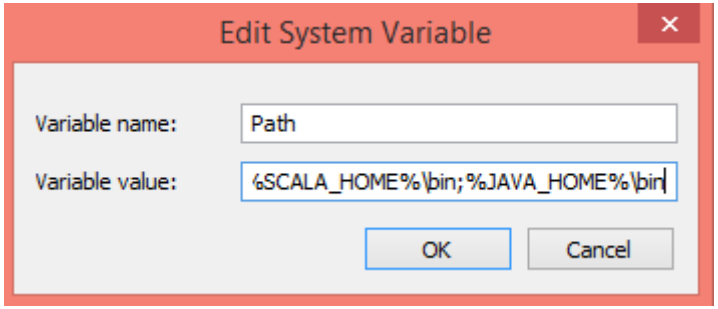
Table of Contents

Exercise 1: Spark Installation: Windows: 2
Install Spark in centos Linux. 9
Exercise : Spark Installation - Redhat Linux. 14
Exercise :Interactive Analysis with the Spark Shell 20
Spark Streaming With Kafka – Java API 23

Exercise 1: Spark Installation: Windows:

You need to install java on your system PATH, or the JAVA_HOME environment variable pointing to a Java installation.




```
C:\Users\henry>echo %JAVA_HOME%
D:\jdk1.8.0_45
```

```
C:\Users\henry>java -version
java version "1.8.0_45"
Java(TM) SE Runtime Environment (build 1.8.0_45-b15)
Java HotSpot(TM) 64-Bit Server VM (build 25.45-b02, mixed mode)
```

Spark runs on Java 6+ and Python 2.6+.

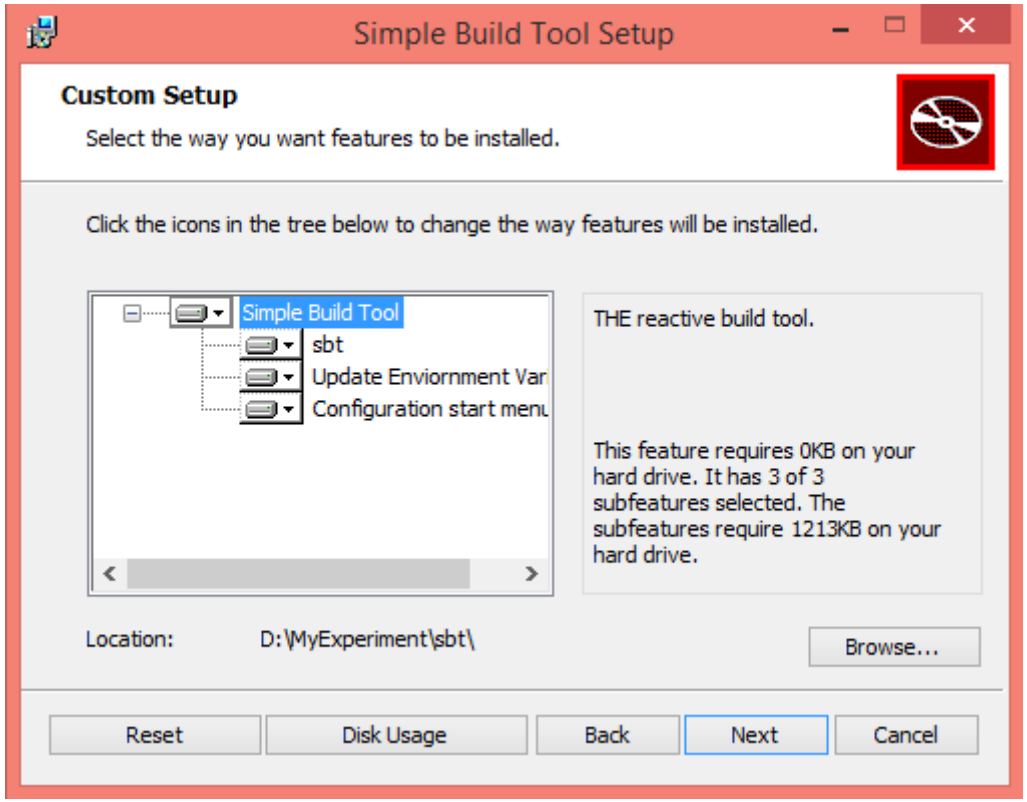
Unzip spark-1.3.0-bin-hadoop2.4.tar to D:\MyExperiment

This PC ▸ New Volume (D:) ▸ MyExperiment ▸ spark-1.3

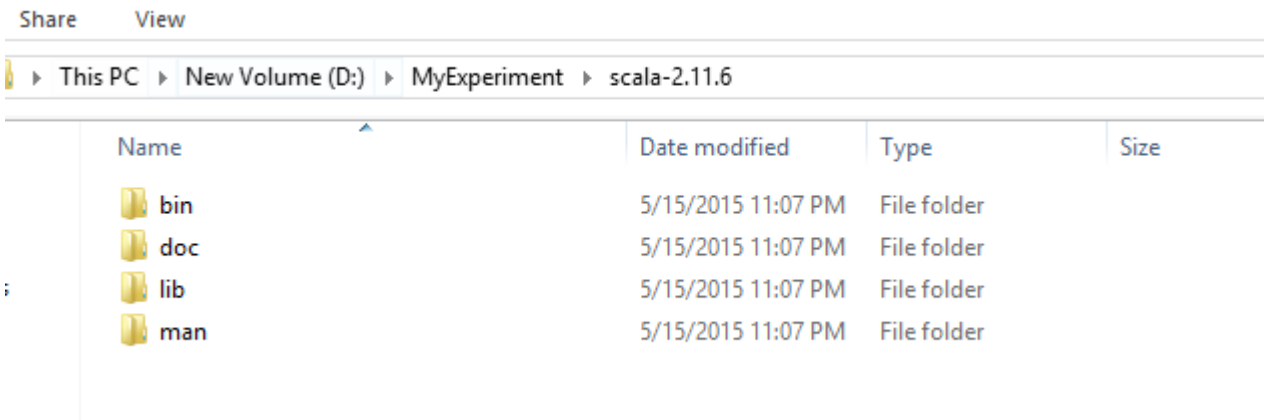
Name	Date modified	Type	Size
bin	5/15/2015 10:46 PM	File folder	
conf	5/15/2015 10:46 PM	File folder	
data	5/15/2015 10:46 PM	File folder	
ec2	5/15/2015 10:46 PM	File folder	
examples	5/15/2015 10:46 PM	File folder	
lib	5/15/2015 10:46 PM	File folder	
python	5/15/2015 10:46 PM	File folder	
sbin	5/15/2015 10:46 PM	File folder	
CHANGES	3/6/2015 6:01 AM	Text Document	245 KB
LICENSE	3/6/2015 6:01 AM	File	46 KB
NOTICE	3/6/2015 6:01 AM	File	23 KB
README.md	3/6/2015 6:01 AM	MD File	4 KB
RELEASE	3/6/2015 6:01 AM	File	1 KB

Install sbt.MSI & execute it.

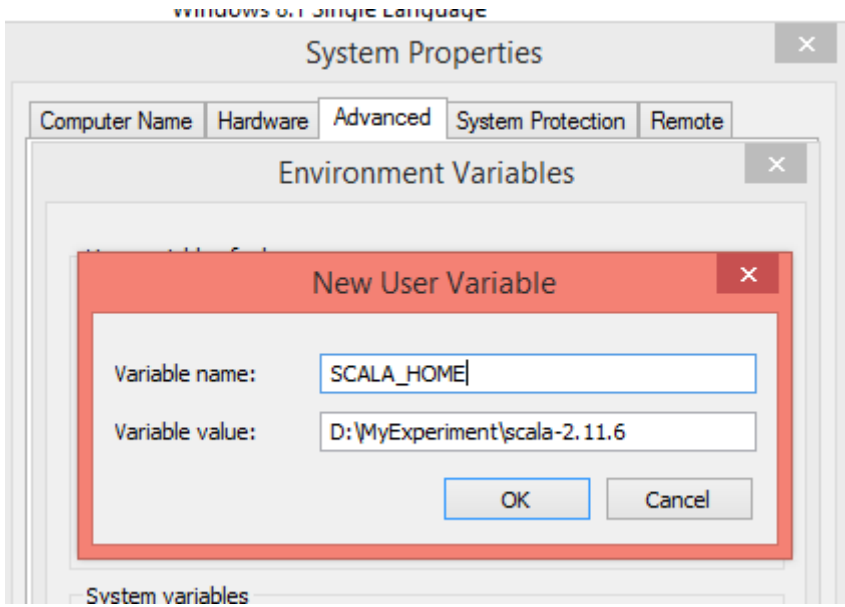
sbt-0.13.8.msi

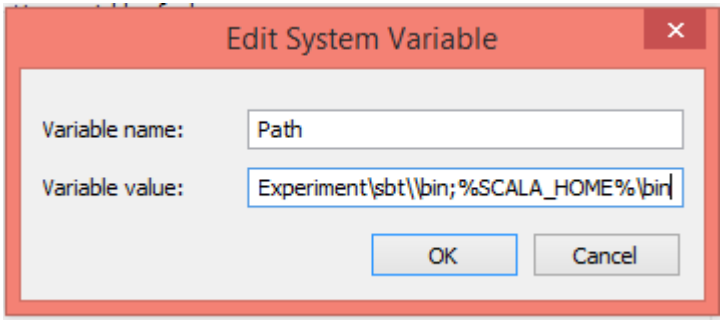


Untar scala-2.11.6



Set SCALA_HOME environment variable & set the PATH variable to the bin directory of scala.






```
C:\Users\henry>scala -version
Scala code runner version 2.11.6 -- Copyright 2002-2013, LAMP/EPFL

C:\Users\henry>
```

change to the installation directory : D:\MyExperiment\spark-1.3>cd bin and execute spark-shell

```
D:\MyExperiment\spark-1.3\bin>spark-shell
log4j:WARN No appenders could be found for logger (org.apache.hadoop.metrics2.li
b.MutableMetricsFactory).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more in
fo.
```

```
15/05/15 23:22:19 INFO Executor: Using REPL class URI: http://192.168.188.1:4967
2
15/05/15 23:22:19 INFO AkkaUtils: Connecting to HeartbeatReceiver: akka.tcp://sp
arkDriver@ht:49686/user/HeartbeatReceiver
15/05/15 23:22:19 INFO NettyBlockTransferService: Server created on 49706
15/05/15 23:22:19 INFO BlockManagerMaster: Trying to register BlockManager
15/05/15 23:22:19 INFO BlockManagerMasterActor: Registering block manager localh
ost:49706 with 265.1 MB RAM, BlockManagerId(<driver>, localhost, 49706)
15/05/15 23:22:19 INFO BlockManagerMaster: Registered BlockManager
15/05/15 23:22:19 INFO SparkILoop: Created spark context..
Spark context available as sc.
15/05/15 23:22:20 INFO SparkILoop: Created sql context (with Hive support)..
SQL context available as sqlContext.

scala>
```

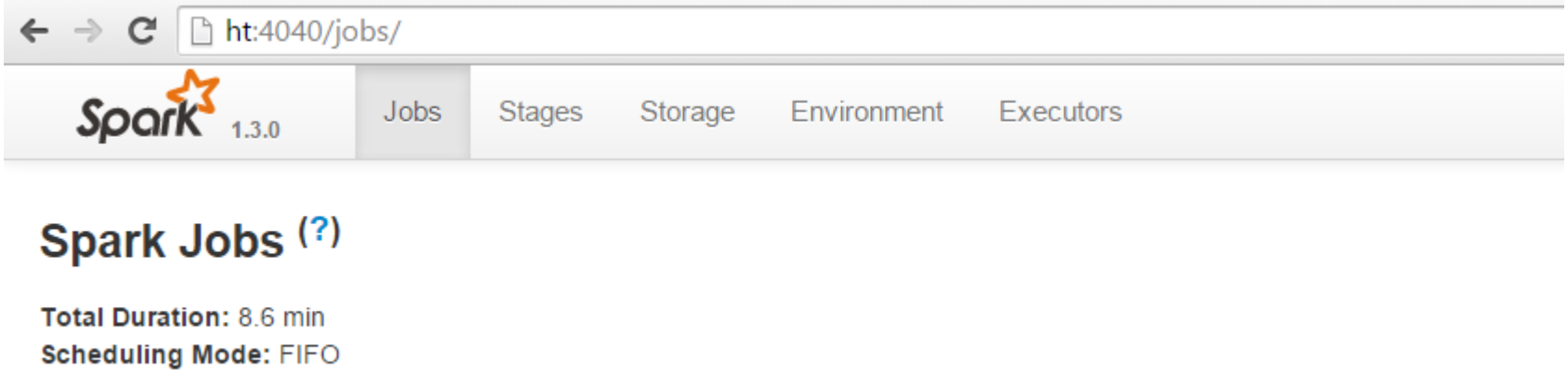
Create a data set of 1...10000 integers

val data = 1 to 10000

```
scala> val data = 1 to 10000
data: scala.collection.immutable.Range.Inclusive = Range(1, 2, 3, 4, 5, 6, 7, 8,
 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48,
49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88,
89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 1
07, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 1
23, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 1
39, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 1
55, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170...

scala>
```

http://ht:4040/jobs/



Install Spark in centos Linux.

Use: centos 7 64 CLI / spark-3.0.0-bin-hadoop3.2.tgz / `scala-2.13.3`

create a folder /opt and untar all related software inside this folder only.

Untar spark to /opt

tar -xvf spark-3.* -C /opt/

it should create a folder inside /opt and rename to spark folder

#mv spark-*/ spark

Untar scala*

#tar -xvf scala* -C /opt

#mv scala* scala

Set the path and variable as follows:

#vi ~/.bashrc or `~/.bashr_profile`

And include the following anywhere in the file.

```
export SCALA_HOME=/opt/scala
export PATH=$PATH:$SCALA_HOME/bin

export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin
```

Type bash, so that variables that were set earlier become effective.

Install JDK and set the PATH variable as shown below if not done earlier else skip this step: (chose 32 or 64 bit depends on your server)

```
tar -xvf jdk-8u45-linux-x64.tar.gz -C /opt

export  JAVA_HOME=/opt/jdk1.8.0_45
export PATH=:$JAVA_HOME/bin :$PATH:$SCALA_HOME/bin
```

Change to the installation directory: /opt/spark/bin and execute

```
./spark-shell
```

```
bash-3.2$ spark-shell
20/08/29 15:17:38 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform.
.. using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://192.168.29.132:4040
Spark context available as 'sc' (master = local[*], app id = local-1598694472393).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 2.4.6
      /_/

Using Scala version 2.11.12 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_171)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

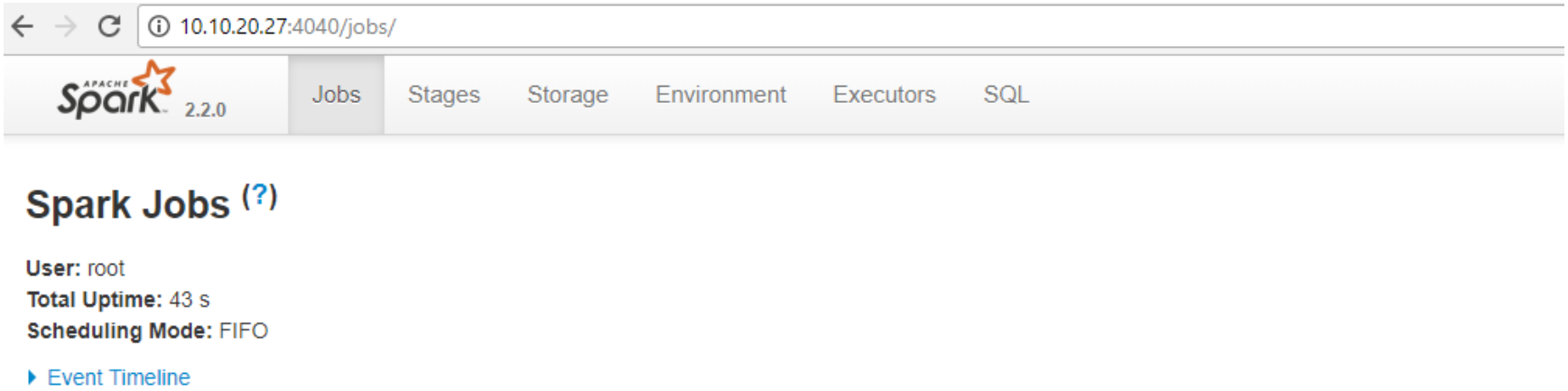
enter the following command in the scala console to create a data set of 1...10000 integers

val data = 1 to 10000

```
scala> val data = 1 to 10000
data: scala.collection.immutable.Range.Inclusive = Range(1, 2, 3, 4, 5, 6, 7, 8,
 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48,
49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88,
89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 1
07, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 1
23, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 1
39, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 1
55, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170...

scala>
```

http://localhost:4040/jobs/



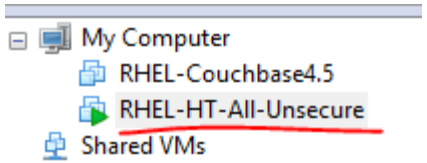
Start in a standalone cluster Mode. Further details on this will be discussed later.

./sbin/start-master.sh

---------- Lab Ends Here ----------

Exercise : Spark Installation - Redhat Linux.

Goals: To install Spark on redhat linux OS.



Ensure to use latest software : spark-2.1.0-bin-hadoop2.7.tgz.tar
You need to replace with the specific version of software wherever relevant.

Create a folder /spark and un tar all related software inside this folder only. So that all installation happens in a single folder for better manageability.

#mkdir /spark

Untar spark-1.3.0-bin-hadoop2.4.tar to /spark

```
spark-1.3.0-bin-hadoop2.4/examples/src/main/resources/kv1.txt
[root@localhost software]# tar -xvf spark-1.3.0-bin-hadoop2.4.tgz -C /spark
```

it should create one folder inside /spark

```
[root@localhost spark]# ls
spark-1.3.0-bin-hadoop2.4
[root@localhost spark]#
```

Untar scala-2.11.6

tar -xvf scala-2.11.6.tgz -C /spark

```
[root@localhost software]#
[root@localhost software]# pwd
/mnt/hgfs/Spark/software
[root@localhost software]# ls
scala-2.11.6.tgz  spark-1.3.0-bin-hadoop2.4.tgz  winutils.exe
[root@localhost software]# tar -xvf scala-2.11.6.tgz -C /spark
```

Set the path and variable as follows: We are including Scala home and bin folder in the path variable.

vi ~/.bashrc
export SCALA_HOME=/spark/scala-2.11.6
export PATH=$PATH:$SCALA_HOME/bin

```
File  Edit  View  Terminal  Tabs  Help
# .bashrc

# User specific aliases and functions

alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'

export SCALA_HOME=/spark/scala-2.11.6
export PATH=$PATH:$SCALA_HOME/bin

# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi
~
```

Type bash

Install JDK as follows and set the PATH with environment as follows: (chose 32 or 64 bit depends on your server)

tar -xvf jdk-8u45-linux-x64.tar.gz -C /spark

```
[root@localhost software]# cd /spark
[root@localhost spark]# pwd
/spark
[root@localhost spark]# ls
jdk1.8.0_45  scala-2.11.6  spark-1.3.0-bin-hadoop2.4
[root@localhost spark]#
```

export JAVA_HOME=/spark/jdk1.8.0_45

export PATH=:$JAVA_HOME/bin :$PATH:$SCALA_HOME/bin

```
# .bashrc

# User specific aliases and functions

alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'

export SCALA_HOME=/spark/scala-2.11.6
export JAVA_HOME=/spark/jdk1.8.0_45
export PATH=$JAVA_HOME/bin:$PATH:$SCALA_HOME/bin
# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi
~
```

Rename the installation folder as shown below: You need to change to installation directory before issuing this command.

#mv spark-2.1.0-bin-hadoop2.7/ spark-2.1/

change to the installation directory : \spark\spark-2.1>cd bin and execute

./spark-shell

```
[root@master spark]# cd spark-2.1/
[root@master spark-2.1]# ls
bin   data      jars     licenses  python  README.md  sbin
conf  examples  LICENSE  NOTICE    R       RELEASE    yarn
[root@master spark-2.1]# bin/spark-shell
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLeve
l(newLevel).
```

```
15/05/15 23:22:19 INFO Executor: Using REPL class URI: http://192.168.188.1:4967
2
15/05/15 23:22:19 INFO AkkaUtils: Connecting to HeartbeatReceiver: akka.tcp://sp
arkDriver@ht:49686/user/HeartbeatReceiver
15/05/15 23:22:19 INFO NettyBlockTransferService: Server created on 49706
15/05/15 23:22:19 INFO BlockManagerMaster: Trying to register BlockManager
15/05/15 23:22:19 INFO BlockManagerMasterActor: Registering block manager localh
ost:49706 with 265.1 MB RAM, BlockManagerId(<driver>, localhost, 49706)
15/05/15 23:22:19 INFO BlockManagerMaster: Registered BlockManager
15/05/15 23:22:19 INFO SparkILoop: Created spark context..
Spark context available as sc.
15/05/15 23:22:20 INFO SparkILoop: Created sql context (with Hive support)..
SQL context available as sqlContext.

scala>
```

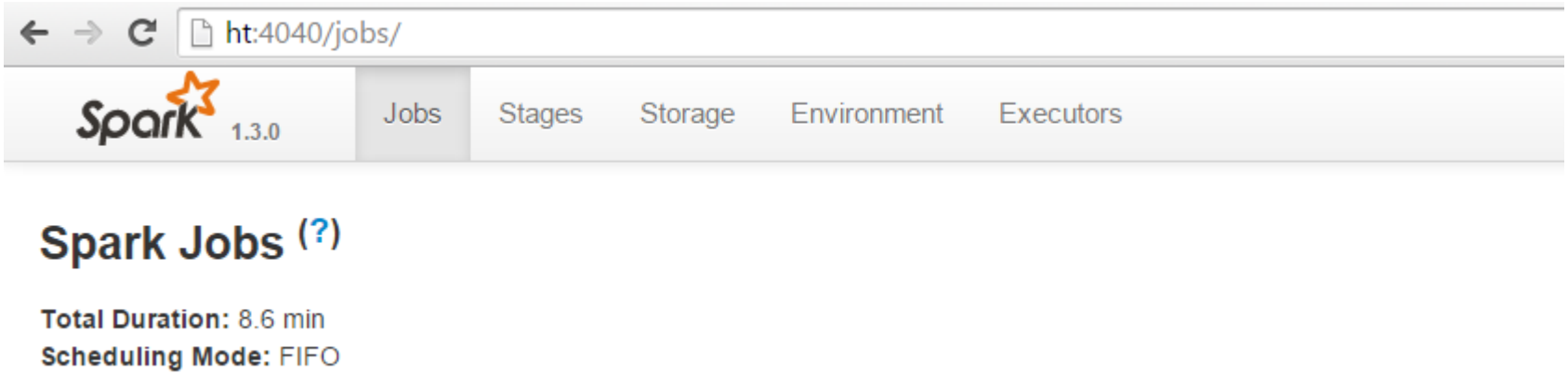
enter the following command in the scala console to create a data set of 1...10000 integers

```
val data = 1 to 10000
```

```
scala> val data = 1 to 10000
data: scala.collection.immutable.Range.Inclusive = Range(1, 2, 3, 4, 5, 6, 7, 8,
 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48,
49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68,
69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88,
89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 1
07, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 1
23, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 1
39, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 1
55, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170...

scala>
```

http://ht:4040/jobs/



Exercise :Interactive Analysis with the Spark Shell

Start the spark shell:

```
./bin/spark-shell
```

```
C:\Windows\system32\cmd.exe - spark-shell
 port 1580.
15/06/03 23:27:33 INFO SparkEnv: Registering OutputCommitCoordinator
15/06/03 23:27:33 INFO Server: jetty-8.y.z-SNAPSHOT
15/06/03 23:27:33 INFO AbstractConnector: Started SelectChannelConnector@0.0.0.0
:4040
15/06/03 23:27:33 INFO Utils: Successfully started service 'SparkUI' on port 404
0.
15/06/03 23:27:33 INFO SparkUI: Started SparkUI at http://ht:4040
15/06/03 23:27:34 INFO Executor: Starting executor ID <driver> on host localhost

15/06/03 23:27:34 INFO Executor: Using REPL class URI: http://192.168.188.1:1566

15/06/03 23:27:34 INFO AkkaUtils: Connecting to HeartbeatReceiver: akka.tcp://sp
arkDriver@ht:1579/user/HeartbeatReceiver
15/06/03 23:27:34 INFO NettyBlockTransferService: Server created on 1599
15/06/03 23:27:34 INFO BlockManagerMaster: Trying to register BlockManager
15/06/03 23:27:34 INFO BlockManagerMasterActor: Registering block manager localh
ost:1599 with 265.1 MB RAM, BlockManagerId(<driver>, localhost, 1599)
15/06/03 23:27:34 INFO BlockManagerMaster: Registered BlockManager
15/06/03 23:27:35 INFO SparkILoop: Created spark context..
Spark context available as sc.
15/06/03 23:27:36 INFO SparkILoop: Created sql context (with Hive support)..
SQL context available as sqlContext.

scala>
```

Running word count:

```
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
```

```
scala> import org.apache.spark.SparkContext
import org.apache.spark.SparkContext

scala> import org.apache.spark.SparkContext._
import org.apache.spark.SparkContext._
```

```
val txtFile = "spam.data"
val txtData = sc.textFile(txtFile)
txtData.cache()
```

```
scala> val txtFile = "spam.data"
txtFile: String = spam.data

scala> val txtData = sc.textFile(txtFile)
15/05/16 16:33:21 INFO MemoryStore: ensureFreeSpace(159214) called with curMem=1
81810, maxMem=278019440
15/05/16 16:33:21 INFO MemoryStore: Block broadcast_1 stored as values in memory
 (estimated size 155.5 KB, free 264.8 MB)
15/05/16 16:33:21 INFO MemoryStore: ensureFreeSpace(22692) called with curMem=34
1024, maxMem=278019440
15/05/16 16:33:21 INFO MemoryStore: Block broadcast_1_piece0 stored as bytes in
memory (estimated size 22.2 KB, free 264.8 MB)
15/05/16 16:33:21 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on l
ocalhost:63788 (size: 22.2 KB, free: 265.1 MB)
15/05/16 16:33:21 INFO BlockManagerMaster: Updated info of block broadcast_1_pie
ce0
15/05/16 16:33:21 INFO SparkContext: Created broadcast 1 from textFile at <conso
le>:27
txtData: org.apache.spark.rdd.RDD[String] = spam.data MapPartitionsRDD[3] at tex
tFile at <console>:27

scala> txtData.cache()
res2: txtData.type = spam.data MapPartitionsRDD[3] at textFile at <console>:27
```

txtData.count()

```
scala> txtData.count()
15/05/16 16:33:43 INFO FileInputFormat: Total input paths to process : 1
15/05/16 16:33:43 INFO SparkContext: Starting job: count at <console>:30
15/05/16 16:33:43 INFO DAGScheduler: Got job 0 (count at <console>:30) with 2 ou
tput partitions (allowLocal=false)
15/05/16 16:33:43 INFO DAGScheduler: Final stage: Stage 0(count at <console>:30)

15/05/16 16:33:43 INFO DAGScheduler: Parents of final stage: List()
15/05/16 16:33:43 INFO DAGScheduler: Missing parents: List()
15/05/16 16:33:43 INFO DAGScheduler: Submitting Stage 0 (spam.data MapPartitions
RDD[3] at textFile at <console>:27), which has no missing parents
15/05/16 16:33:43 INFO MemoryStore: ensureFreeSpace(2640) called with curMem=363
716, maxMem=278019440
15/05/16 16:33:43 INFO MemoryStore: Block broadcast_2 stored as values in memory
 (estimated size 2.6 KB, free 264.8 MB)
15/05/16 16:33:43 INFO MemoryStore: ensureFreeSpace(1929) called with curMem=366
356, maxMem=278019440
15/05/16 16:33:43 INFO MemoryStore: Block broadcast_2_piece0 stored as bytes in
memory (estimated size 1929.0 B, free 264.8 MB)
15/05/16 16:33:43 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on l
ocalhost:63788 (size: 1929.0 B, free: 265.1 MB)
15/05/16 16:33:43 INFO BlockManagerMaster: Updated info of block broadcast_2_pie
ce0
15/05/16 16:33:43 INFO SparkContext: Created broadcast 2 from broadcast at DAGSc
heduler.scala:839
15/05/16 16:33:44 INFO DAGScheduler: Submitting 2 missing tasks from Stage 0 (sp
am.data MapPartitionsRDD[3] at textFile at <console>:27)
15/05/16 16:33:44 INFO TaskSchedulerImpl: Adding task set 0.0 with 2 tasks
15/05/16 16:33:44 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0, lo
calhost, PROCESS_LOCAL, 1309 bytes)
15/05/16 16:33:44 INFO TaskSetManager: Starting task 1.0 in stage 0.0 (TID 1, lo
calhost, PROCESS_LOCAL, 1309 bytes)
15/05/16 16:33:44 INFO Executor: Running task 0.0 in stage 0.0 (TID 0)
15/05/16 16:33:44 INFO Executor: Running task 1.0 in stage 0.0 (TID 1)
15/05/16 16:33:44 INFO CacheManager: Partition rdd_3_1 not found, computing it
15/05/16 16:33:44 INFO CacheManager: Partition rdd_3_0 not found, computing it
15/05/16 16:33:44 INFO HadoopRDD: Input split: file:/D:/MyExperiment/spark-1.3/b
in/spam.data:0+349170
```

Output:

```
15/05/16 16:33:45 INFO DAGScheduler: Job 0 finished: count at <console>:30, took
 1.728775 s
res3: Long = 4601

scala>
```

Spark Streaming With Kafka – Java API

Goals: To integrate Apache Kafka with Spark Using Streaming API.

You need to complete the Lab of Kafka installation.

Steps to be performed:

- Install Kafka and start it along with producer.
- Write java program to connect to Kafka and get the data from Kafka using streaming
- Execute it to fetch the data.

Start zookeeper and the first Broker using terminal

#sh startABroker.sh

Keep thes console open.

Create a topic

Let's create a topic named "test" with a single partition and only one replica: Open one more terminal

#/opt/kafka/bin/kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1 --topic test

```
Telnet 192.168.188.134
[root@ht kafka_2.11-0.8.2.1]# pwd
/spark/kafka_2.11-0.8.2.1
[root@ht kafka_2.11-0.8.2.1]#
[root@ht kafka_2.11-0.8.2.1]#
[root@ht kafka_2.11-0.8.2.1]#
[root@ht kafka_2.11-0.8.2.1]# pwd
/spark/kafka_2.11-0.8.2.1
[root@ht kafka_2.11-0.8.2.1]# bin/kafka-topics.sh --create --zookeeper localhos
t:2181 --replication-factor 1 --partitions 1 --topic test
Created topic "test".
[root@ht kafka_2.11-0.8.2.1]#
```

We can now see that topic if we run the list topic command:

>/opt/kafka/**bin/kafka-topics.sh --list --zookeeper localhost:2181**

```
[root@ht kafka_2.11-0.8.2.1]# bin/kafka-topics.sh --list --zookeeper localhost:
2181
test
[root@ht kafka_2.11-0.8.2.1]#
```

Send some messages

// Open a new console, change directory to the kafka installation folder

```
test
[root@ht kafka_2.11-0.8.2.1]# bin/kafka-console-producer.sh --broker-list local
host:9092 --topic test
[2015-06-10 00:25:31,582] WARN Property topic is not valid (kafka.utils.Verifiab
leProperties)
I am sending a message
```

#/opt/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092 --topic test

// type some message

- Test Message
- Kafka Message Kafka
- Windows Kafka

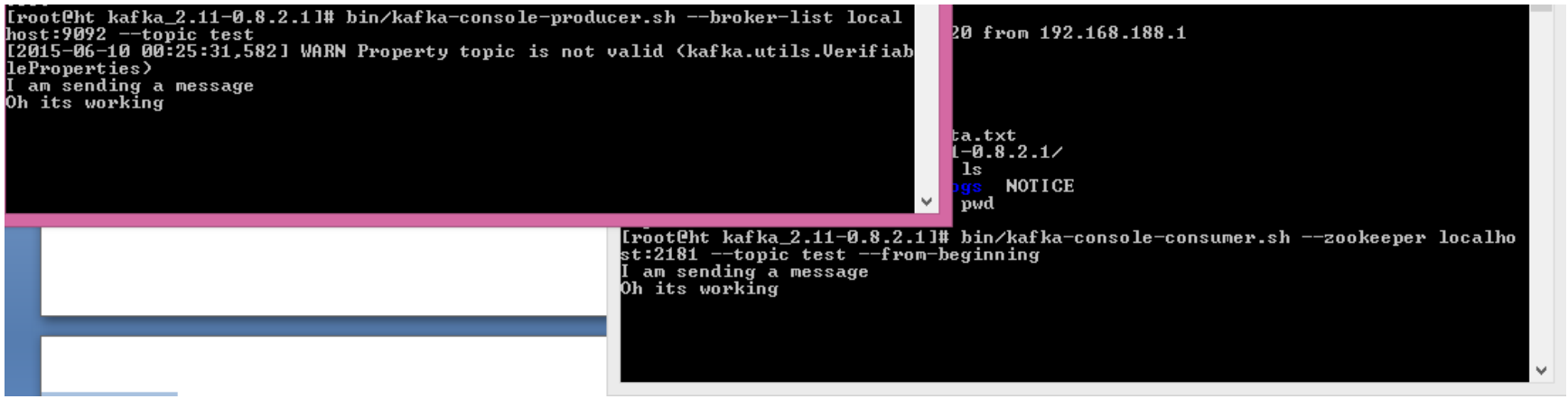
Start a consumer

// Open a new console, change directory to the kafka installation folder

#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test --from-beginning

```
(base) Henrys-MacBook-Air:~ henrypotsangbam$
(base) Henrys-MacBook-Air:~ henrypotsangbam$
(base) Henrys-MacBook-Air:~ henrypotsangbam$ /opt/kafka/bin/kafka-console-consumer.sh --bootstra
p-server localhost:9092 --topic test --from-beginning
Test Message
```

Try sending some information from the producer to consumer.



If you are able to see all the messages type in the producer in the consumer console then , You have successfully configured Kafka, now let us fetch information using spark.

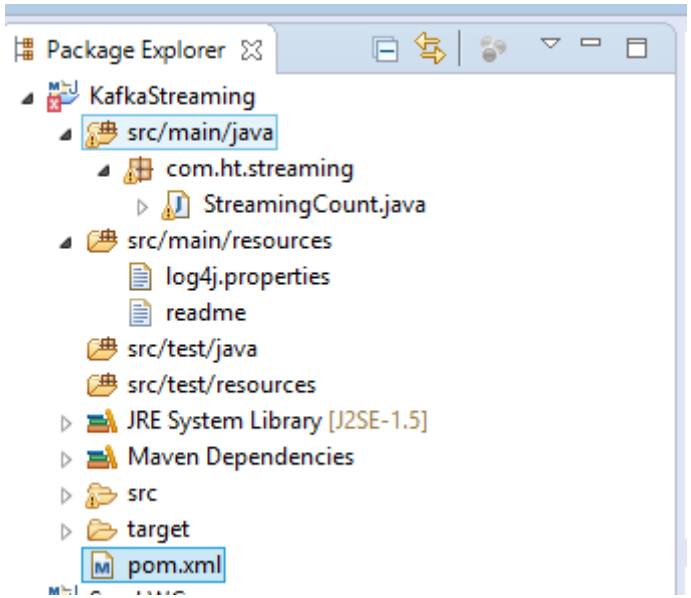
Do not close any of the above window.

Create a maven project on your desktop/laptop

Maven project details:

- create simple project
- groupId:com.ht
- artifactId:KafkaStreaming
- version:0.0.1-SNAPSHOT

You should get as follows after that.



Create the followings:

- package : com.ht.streaming
- java class: StreamingCount
- log4j.properties

Update the file as follows:

pom.xml

```
package com.tos.streamming;

import java.util.Arrays;

import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Encoders;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.SparkSession;
import org.apache.spark.sql.execution.streaming.ProcessingTimeTrigger;
import org.apache.spark.sql.streaming.StreamingQuery;

public class StreamingCount {

     public static void main(String[] args1) {
          String master ="local[2]";
          SparkSession spark =
SparkSession.builder().master(master).appName("JavaStructuredWordCount").getOrCreate();

          // Subscribe to 1 topic
          Dataset<Row> df =
spark.readStream().format("kafka").option("kafka.bootstrap.servers", "localhost:9092")
                    .option("subscribe", "test").load();
          df = df.selectExpr(  "CAST(value AS STRING)");
        df.printSchema();
          // Split the lines into words
          Dataset<String> words = df.as(Encoders.STRING()).flatMap(
                    (FlatMapFunction<String, String>) x -> Arrays.asList(x.split(" 
")).iterator(), Encoders.STRING());
```

```
        // Generate running word count
        Dataset<Row> wordCounts = words.groupBy("value").count();
        wordCounts.printSchema();
        // Start running the query that prints the running counts to the console
        StreamingQuery query = null;
        try {

            query =
                    wordCounts
                      .selectExpr( "CAST(value  AS STRING) as key", "CAST (count as
STRING) as value")
                      .writeStream()
                      .format("kafka")
                      .option("kafka.bootstrap.servers", "localhost:9092")
                      .option("topic", "mytopic")
                      .option("checkpointLocation", "/tmp/test")
                      .outputMode("update")
                      .trigger( ProcessingTimeTrigger.ProcessingTime("5 seconds"))
                      .start();

            query.awaitTermination();
        } catch (Exception e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

    }

}
```

StreamingCount program will consume message from topic, test and count the occurrences of the word in each line and publish message to the topic, mytopic with word as the key and count or the occurrence of the word as the value of the message.

Create a class and replace with the following code:

```
package com.tos.streamming;

import java.util.Arrays;

import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Encoders;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.SparkSession;
import org.apache.spark.sql.execution.streaming.ProcessingTimeTrigger;
import org.apache.spark.sql.streaming.StreamingQuery;

public class StreamingCount {

     public static void main(String[] args1) {
          String master ="local[2]";
          SparkSession spark =
SparkSession.builder().master(master).appName("JavaStructuredWordCount").getOrCreate();

          // Subscribe to 1 topic
          Dataset<Row> df =
spark.readStream().format("kafka").option("kafka.bootstrap.servers", "localhost:9092")
                    .option("subscribe", "test").load();
          df = df.selectExpr(  "CAST(value AS STRING)");
        df.printSchema();
```

```
        // Split the lines into words
        Dataset<String> words = df.as(Encoders.STRING()).flatMap(
                  (FlatMapFunction<String, String>) x -> Arrays.asList(x.split(" 
")).iterator(), Encoders.STRING());
        // Generate running word count
        Dataset<Row> wordCounts = words.groupBy("value").count();
        wordCounts.printSchema();
        // Start running the query that prints the running counts to the console
        StreamingQuery query = null;
        try {

            query =
                      wordCounts
                        .selectExpr( "CAST(value  AS STRING) as key", "CAST (count as 
STRING) as value")
                        .writeStream()
                        .format("kafka")
                        .option("kafka.bootstrap.servers", "localhost:9092")
                        .option("topic", "mytopic")
                        .option("checkpointLocation", "/tmp/test")
                        .outputMode("update")
                        .trigger( ProcessingTimeTrigger.ProcessingTime("5 seconds"))
                        .start();

            query.awaitTermination();
        } catch (Exception e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

    }

}
```

Update log4j.properties

```
# Set everything to be logged to the console
log4j.rootCategory=ERROR, console
log4j.appender.console=org.apache.log4j.ConsoleAppender
log4j.appender.console.target=System.err
log4j.appender.console.layout=org.apache.log4j.PatternLayout
log4j.appender.console.layout.ConversionPattern=%d{yy/MM/dd HH:mm:ss} %p %c{1}: %m%n

# Settings to quiet third party logs that are too verbose
log4j.logger.org.eclipse.jetty=WARN
log4j.logger.org.eclipse.jetty.util.component.AbstractLifeCycle=ERROR
log4j.logger.org.apache.spark.repl.SparkIMain$exprTyper=INFO
log4j.logger.org.apache.spark.repl.SparkILoop$SparkILoopInterpreter=INFO
```

Run the java program.

Right click on StreamingCount --> Run as --> Java Application

Send some text from the producer console of the Kafka.

```
#/opt/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092 --topic test
```

```
[root@ht kafka_2.11-0.8.2.1]# bin/kafka-console-producer.sh --broker-list local
host:9092 --topic test1
[2015-06-17 23:55:31,960] WARN Property topic is not valid (kafka.utils.Verifiab
leProperties)
Sending information to kafka
```

You wil get the response as follows: Open a terminal to consume message that is being publish to mytopic after transformation by Spark.

```
#/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic mytopic --
from-beginning --property print.key=true
```

```
DOne     1
finally  1
works..  1
Then     1
Great    2
is       -662880094343107518
A        1
WOW      1
that     1
is       -662880094343107517
is       -662880094343107516
works    2
us       3
Message  1
check    1
Let      1
kafka    3
Message  2
Sending  2
Kafka    2
to       2
```

```
scripts — java -Xmx512M -server -XX:+UseG1GC -XX:MaxGCPauseMillis=20 -...
(base) Henrys-MacBook-Air:scripts henrypotsangbam$ ls
ca-cert                          docker
ca-cert.srl                      server.keystore.jks
ca-key                           server.truststore.jks
cert-file                        start3Brokers.sh
cert-signed                      startABroker.sh
client.truststore.jks            startAll.sh
consumer.properties              stopBroker.sh
consumer1.properties             stopZookeeper.sh
custom-reassignment.json
(base) Henrys-MacBook-Air:scripts henrypotsangbam$ /opt/kafka/bin/kafka-console
producer.sh --broker-list localhost:9092 --topic test
>Sending messages to kafka
>kafka stransform
>kafka
>Sending Message to Kafka
>
```

Congrats! You have successfully consume messages from the Kafka using spark, perform transformation and publish the transform message to the topic for further data pipeline.

---------- Lab Ends Here ----------